ECE3300L UART Project - Part 1

UART Transmitter Core: 115200 Baud, 8-N-1

Project direction
This is the protocol foundation of an expanding RTL project. In this part, you will create and connect reusable UART timing and transmit modules. A board-level top module, pin constraints, display logic, and application-specific message controller are intentionally deferred to a later part.

What you will build

1. An enabled, synchronous baud-tick generator derived from the 100 MHz FPGA clock.
2. An 8-bit UART transmitter that produces a standard asynchronous serial frame.
3. A request interface that accepts one byte when the transmitter is idle.
4. Status outputs that identify when transmission is active and when a frame has completed.
5. A self-contained transmitter core that can later be instantiated by a top module, message controller, FIFO, or processor interface.

UART configuration for Part 1

Setting	Required value	Meaning
Baud rate	115200 baud	One serial bit is transmitted every 1/115200 second.
Data length	8 bits	Each frame carries one byte.
Parity	None	No parity bit is inserted after the data bits.
Stop bits	1	The line is held high for one bit period after D7.
Bit order	LSB first	D0 is transmitted before D1 through D7.
Idle level	Logic 1	The serial output remains high between frames.

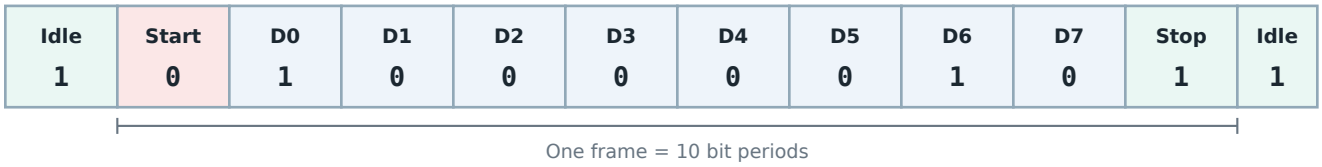
Lab objectives

- Understand how a UART byte is framed without a transmitted clock.
- Translate baud rate into an exact number of FPGA clock cycles per serial bit.
- Use a finite-state machine to control start, data, and stop intervals.
- Preserve the input byte internally while its bits are shifted out.
- Create clean module boundaries that support later project expansion.

UART protocol and timing

The receiver has no copy of the FPGA clock. It reconstructs each byte from the TX transition, the agreed baud rate, and the agreed frame format.

Example frame: ASCII 'A' = 0x41 (8-N-1, LSB first)



The falling edge from idle high to the start bit marks the beginning of a frame. Every following field must remain stable for exactly one bit period. After D7, the stop bit returns TX high.

Baud-rate calculation

Clock cycles per bit
For a 100 MHz system clock and 115200 baud: `100,000,000 / 115,200 = 868.055...`. Use **`868 clock cycles per bit`**. This produces approximately 115207 baud, an error of only +0.0064%.

Quantity	Calculation	Approximate result
System-clock period	1 / 100 MHz	10 ns
UART bit period	1 / 115200	8.68 us
8-N-1 frame length	1 start + 8 data + 1 stop	10 bits
Time per byte	10 / 115200	86.8 us
Maximum payload rate	115200 / 10	11520 bytes/s

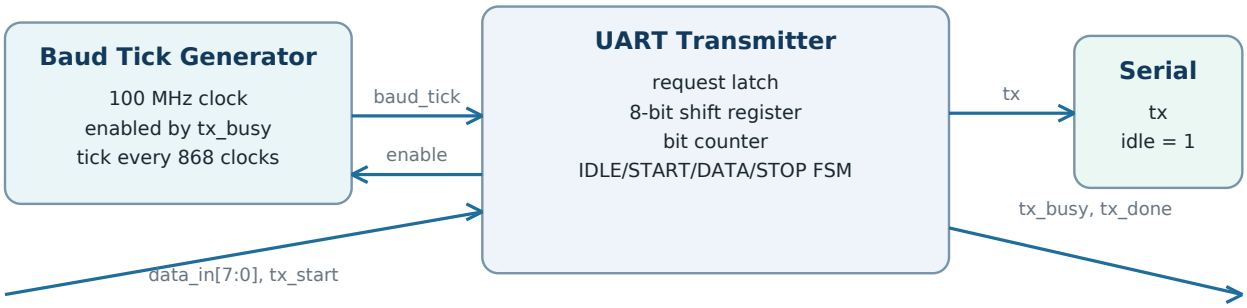
Core module interfaces

Module	Signal	Direction	Purpose
uart_baud_tick	clk	Input	100 MHz system clock.
uart_baud_tick	rst_n	Input	Active-low reset for the timing counter.
uart_baud_tick	enable	Input	High while a frame is active; low clears and aligns the counter.
uart_baud_tick	baud_tick	Output	One system-clock-wide pulse every 868 enabled clocks.
uart_tx	clk, rst_n	Inputs	Shared clock and active-low reset.
uart_tx	baud_tick	Input	Permission to advance to the next UART bit.
uart_tx	data_in[7:0]	Input	Byte requested for transmission.
uart_tx	tx_start	Input	One-cycle request, accepted only while idle.
uart_tx	tx	Output	Serialized UART line; high while idle.
uart_tx	tx_busy	Output	High from accepted request through the stop bit.
uart_tx	tx_done	Output	One-cycle completion pulse after the frame.

Start-bit alignment
Keep the baud counter cleared while `enable` is low. Connect `tx_busy` to `enable`. When a request is accepted, TX goes low and the first `baud_tick` arrives only after 868 complete system-clock cycles.

Module-level architecture

This part ends at the reusable UART core. The future project top will decide where bytes come from and which physical pin carries TX.



Reset and system clock are shared by both modules.

The future top module will connect these blocks to board pins and higher-level message logic.

Transmitter state sequence

State	TX value	Duration	Required behavior
IDLE	1	Indefinite	Wait for tx_start. Accept a request only when tx_busy is low.
START	0	1 bit period	Create the falling edge that synchronizes the receiver.
DATA	D0 through D7	8 bit periods	Present one latched data bit per baud_tick, least-significant bit first.
STOP	1	1 bit period	Complete the 8-N-1 frame and return the line to idle.

Request and status behavior

1. The source places a byte on `data_in[7:0]` while `tx_busy` is low.
2. The source pulses `tx_start` for one system-clock cycle.
3. The transmitter immediately copies the byte into an internal register. Later changes to `data_in` must not alter the active frame.
4. The transmitter asserts `tx_busy` and sends START, eight DATA bits, and STOP.
5. After the complete stop-bit interval, the transmitter deasserts `tx_busy` and pulses `tx_done` for one system-clock cycle.

Required design rules

- TX must reset to logic 1. A low idle line would look like a continuous start condition.
- After a request enters START, all remaining serial timing and data-bit advances occur only on `baud_tick`.
- The baud counter counts enabled system-clock cycles; it does not create a second clock.
- Hold the baud counter at zero while `tx_busy` is low so every frame begins with a full-length start bit.
- A new `tx_start` request while busy may be ignored in Part 1. Buffering will be added later if needed.
- Back-to-back bytes are allowed once the previous stop bit is fully complete.

> Scope boundary
> Do not create a top module in this part. Do not assign a package pin, connect a push-button, add a seven-segment display, or build a multi-byte message controller yet.

Step-by-step implementation

Implement the design in the order below. Keep the modules separate so the transmitter can be reused by later project stages.

1. Create uart_baud_tick.v

1. Define parameters for the system clock frequency and target baud rate. For this part, use 100 MHz and 115200 baud.
2. Determine the integer terminal count of 868 system-clock cycles per UART bit.
3. Add an `enable` input and create a counter wide enough to represent values from 0 through 867.
4. On reset or while `enable` is low, clear both the counter and `baud_tick`. This aligns the timing of every new frame.
5. While enabled, keep `baud_tick` low by default. When the counter reaches its terminal count, clear the counter and pulse `baud_tick` high for that one system-clock cycle.

2. Create uart_tx.v

1. Declare the request, timing, serial-output, and status signals listed in the interface table.
2. Define four transmitter states: IDLE, START, DATA, and STOP.
3. Add an 8-bit internal data register and a 3-bit data index. The index must identify D0 through D7.
4. Reset the module to IDLE with TX high, `tx_busy` low, `tx_done` low, the data register cleared, and the data index cleared.

3. Accept and preserve a transmit request

1. While in IDLE, keep TX high and wait for `tx_start`.
2. When a request is accepted, copy `data_in` into the internal register, assert `tx_busy`, clear the data index, and move to START. The asserted busy signal enables the baud counter from a known zero phase.
3. Do not continuously read `data_in` during a frame. Only the saved byte may control the transmitted data bits.

4. Generate the UART frame

1. In START, drive TX low immediately after accepting the request. Remain there until the first `baud_tick`, which now occurs after one complete bit period.
2. In DATA, drive TX from the saved byte at the current data index. On each `baud_tick`, advance to the next bit.
3. After D7 has occupied one complete bit period, move to STOP rather than wrapping the data index.
4. In STOP, drive TX high for one complete bit period. Then return to IDLE, clear `tx_busy`, and pulse `tx_done`.

5. Prepare the core for later expansion

1. Keep clock-rate and baud-rate values parameterized rather than scattering numeric constants through the modules.
2. Use the one-cycle `tx_start` / `tx_done` convention consistently so a future controller can sequence multiple bytes.
3. Place only `uart_baud_tick.v` and `uart_tx.v` in the Part 1 design scope. Top-level integration belongs to the next project stage.

End of Part 1
At this point, the design specification contains a complete single-byte UART transmit path. Testing, board integration, physical pin mapping, multi-byte messages, buffering, and receive logic are outside this handout.